



BOOTCAMP FRONTEND & REACT · SEMANA 4

El contrato

Clase 12 — TypeScript aplicado a TechCart: tipos, interfaces, uniones, genéricos y las tres aduanas. El puente a React.

4 h 30 · L·M·V

Proyecto TechCart

STEERCL

Agenda

1. Punto de partida + cerrar JS

24'

document.title y some/every

2. Por qué TypeScript

26'

los tres fantasmas · qué es en serio

3. Instalar y compilar

32'

npm, tsc, tsconfig, --watch

4. El contrato

38'

interface Producto + el adaptador

5. Lo que React va a pedir

46'

uniones · funciones · <> · narrowing

6. Las tres aduanas

40'

any vs unknown · API · storage · DOM

7. Genérico propio + checkout

40'

la última deuda, ya en TypeScript

8. Cierre + 5 tareas

24'

y el anticipo a React

**Receso de 15 min** entre el bloque 4 y el 5 · naranja = **cierre de JavaScript** · verde = **lo que nunca se dictó**

LA CLASE PASADA SE CORTÓ EN EL BLOQUE 7

Tres cosas quedaron. **Dos son de cinco minutos.**

`document.title`

el contador en la pestaña. Tres líneas, ahora, en JavaScript.

`some / every`

las dos que faltaban de la familia. El botón "Confirmar" se apaga solo.

~~`el checkout`~~

→ hoy, pero escrito en **TypeScript**

es el mejor ejemplo del día de para qué sirve.

Y todos partimos del **mismo código**: descarguen `clase-12/`. El que tenga el suyo más avanzado, que lo guarde aparte y compare al final.

LOS TRES LOS VIVIMOS LA CLASE PASADA, EN VIVO

Los tres eran **el mismo error**

undefined

el respaldo no tenía marca, y la tarjeta la pintó igual.

S/ NaN

el carrito guardado no tenía cantidad, y alguien lo multiplicó.

.toLowerCase()

si marca venía vacía, el buscador moría en la primera tecla.

Un objeto que no tenía la forma que el código esperaba. Y en los tres, **nadie avisó**.

¿Dónde está escrito qué es un producto de TechCart?

en cuatro archivos... y en mi cabeza

EL MODELO MENTAL DE TODO EL DÍA

TypeScript **no se ejecuta**

lo que escribimos · .ts

```
interface Producto {  
  id: number  
  nombre: string  
}  
  
const formatear = (p: Producto) =>  
  `${p.nombre}`
```

→
tsc

lo que ve el navegador · .js

```
// la interface: desapareció entera  
  
const formatear = (p) =>  
  `${p.nombre}`
```

⚠ **Los tipos desaparecen.** De ahí salen las dos consecuencias del día: **(1)** TypeScript no hace **nada** en tiempo de ejecución — no valida la API, no revisa el localStorage; **(2)** sirve para que el editor te avise **antes de correr**. **Te protege de vos mismo, no del mundo exterior.**

LA DIFERENCIA QUE MÁS LOS VA A CHOCAR

Java mira el **nombre**. TypeScript mira la **forma**.

JAVA • TIPADO NOMINAL

```
class Producto implements Vendible {...}  
  
// si no lo DECLARA, no lo es –  
// aunque tenga los mismos campos
```

Manda el **nombre**.

VS

TYPESCRIPT • TIPADO ESTRUCTURAL

```
const suelto = { x: 5, y: 9, color:  
  "rojo" }  
  
const b: Punto = suelto // ✅ entra  
// nunca dijo ser un Punto... pero TIENE  
la forma
```

Manda la **forma**.

¿Por qué así? Porque en JavaScript los objetos **vienen de afuera** —de un fetch, del storage, de un formulario— y ninguno trae etiqueta con su clase. Lo único que tienen es una **forma**.

LOS MISMOS TRES DE TODOS LOS PROYECTOS DE SU VIDA

Tres comandos

`npm init -y`crea **package.json**: la ficha del proyecto y **de qué depende**`npm i -D typescript`baja TypeScript **a este proyecto**. El -D = "para desarrollar, no para publicar" — el navegador nunca lo ve`npx tsc --init`crea **tsconfig.json**: cómo compilo. Borramos casi todo y dejamos 7 líneas`src/*.ts`

lo que escribimos

`npm run dev`

tsc --watch

`dist/*.js`

lo que carga el HTML

Todo JavaScript válido es TypeScript válido. No hay que reescribir nada: se renombra `.js` → `.ts` y se agregan tipos de a poco.

VIENEN DE JAVA: ESTO YA LO SABEN

No anoten lo obvio

```
string // en minúscula, no String
number // UNO solo: no int/long/double
boolean
null · undefined // acá SÍ son tipos
Producto[] // un array de productos
```

```
const nombre: string = "Ana" // de más
const nombre = "Ana" // ✅ ya sabe que es
string
```

```
// se anota lo que NO puede adivinar:
const formatearPrecio = (precio: number):
string =>
`S/ ${precio.toFixed(2)}`
```

Se llama **inferencia**. Si su código se está llenando de : string, están anotando de más: los **parámetros** sí, el resto se deduce.

LA RESPUESTA A LA PREGUNTA DE HACE UNA HORA

Qué es un producto: **en un solo lugar**

api.ts · mapearProducto
datos.ts · el respaldo
ui.ts · tarjetaProducto
main.ts · el buscador



```
export interface Producto {  
  id: number  
  nombre: string  
  precio: number  
  categoria: string  
  stock: number  
  imagen: string  
  marca: string  
  valoracion: number  
}
```

tipos.ts · y al compilar **desaparece entero**

Antes: cuatro copias que **tenían que coincidir** y nadie vigilaba. En React serían veinticinco.

TODO LO DE ESTA DIAPO LO VAN A ESCRIBIR EN REACT

El `|` se lee "o". La **flecha** es una función.

```
type EstadoCarga = "cargando" | "listo" | "error"

// un ítem ES un producto Y ADEMÁS una cantidad
type ItemCarrito = Producto & { cantidad: number }

// una FUNCIÓN también tiene tipo
type AlAgrega = (id: number) => void
```

```
// ...y así se ve una prop de React
interface PropsTarjeta {
  producto: Producto
  onAgregar: (id: number) => void
  destacada?: boolean
}
```

JAVA

enum (crea una clase en runtime) ·
Consumer<T> · Function<T,R>



TYPESCRIPT

La unión **no crea nada**: queda el texto pelado. Y no hay que buscar qué interfaz funcional usar: **se escribe la firma**. void = "no devuelve nada útil".

LO MÁS ELEGANTE DE TYPESCRIPT

El compilador **sigue tus if**

```
interface Producto { ... descripcion?: string }
// el ? significa: PUEDE NO ESTAR

descripcion.slice(0,80)
// ✗ 'descripcion' is possibly 'undefined'

if (descripcion) {
  descripcion.slice(0,80) // ✔ acá adentro ya es
  string
}
```

Afuera del if: `string | undefined`. Adentro: `string`. Sin
castear ni jurar nada.

1 · el if — honesto

y el `?.` / `??` de la clase 8 también valen como prueba

2 · `typeof` · `Array.isArray` · `instanceof`

las tres formas de estrechar. La última: `evento.target instanceof HTMLInputElement`

⚠ El **!** no es una de ellas. `descripcion!.slice()` no revisa nada: **apaga el aviso**. Si te equivocás, revienta igual que antes — pero ahora con el compilador diciendo que estaba todo bien.

Un tipo que recibe otro tipo

Array<Producto>

un array **de productos** · es lo mismo que Producto[]

Promise<Producto[]>

una promesa **de** un array de productos · el await la desenvuelve

useState<ItemCarrito[]>

la primera línea que van a escribir **el lunes**

La caja es la misma; **lo que cambia es qué guarda**. Por ahora basta con leerlos: escribir uno propio es el bloque 7.

Pick<Producto, "id" | "nombre">

solo esos campos

Omit<Producto, "descripcion">

todos **menos** ese

Partial<Producto>

todos **opcionales**

Si mañana Producto gana un campo, **los derivados se actualizan solos**. Una sola fuente de verdad, otra vez.

BLOQUE 6 · DONDE TYPESCRIPT SE GANA EL SUELDO

any apaga TypeScript. **unknown** te obliga a mirar.

any

```
const x: any = "un texto"
x.toFixed(2) // ✅ compila... ✨ revienta
```

"apagá TypeScript acá". Acepta cualquier cosa y te deja hacerle cualquier cosa. Es un **agujero** en el contrato.

unknown

```
const x: unknown = "un texto"
x.toFixed(2) // ❌ el editor te frena
if (typeof x === "number") { ... } // ✅
```

"no sé qué es" — igual que any— **"pero no lo tocás hasta que me demuestres qué es"**.

any es una decisión, no un descuido. Y si es porque no sabés qué tipo poner, la respuesta casi siempre es unknown.

POR ACÁ ENTRA TODO LO QUE NO ESCRIBIMOS NOSOTROS

Anotar un tipo **no valida nada**



La API

`response.json() → any`

✅ aduana: el adaptador



El localStorage

`JSON.parse() → any`

✅ aduana: unknown + isArray



El formulario

`FormDataEntryValue`

✅ aduana: leerCliente()

⚠️ **TypeScript no revisa ninguna de las tres en runtime** — los tipos ya desaparecieron. Si escribís `const p: Producto[] = await r.json()`, TypeScript se queda **feliz...** y si el servidor manda otra cosa, revienta igual que la semana pasada. **La anotación es un deseo; la aduana es la que lo cumple.**

TypeScript te cuida de vos mismo, no del mundo exterior.

LEERLOS YA SABEN · AHORA, ESCRIBIR EL TUYO

La función que ya escribimos **sin explicar**

```
const pedirElemento = <T extends Element>
(selector: string): T => {
  const el = document.querySelector<T>(selector)
  if (!el) throw new Error(`No encontré
"${selector}"`)
  return el
}

const buscador = pedirElemento<HTMLInputElement>
("#buscador")
buscador.value // ✅ un Element pelado NO tiene
.value
```

<T extends...> declara el tipo-parámetro y le pone un **límite**

querySelector<T> se lo **pasa** al genérico de adentro

: T devuelve **lo mismo** que le pidieron

¿Por qué no poner Element y listo? Porque perdería el .value y el .disabled. El genérico **conserva** el tipo en vez de aplastarlo: **quien llama decide qué recibe**.

EN LA PÁGINA DESDE LA CLASE 1 · HOY, Y YA TIPADO

Lo último que le falta a la tienda

1

submit

+
preventDefault():
si no, recarga y se
pierde todo

2

¿es válido?

el navegador ya frenó
lo inválido: **cero**
líneas nuestras

3

leer 9 campos

Object.fromEntries(new
FormData(f))

4

guardar el Pedido

clave **aparte**:
techcart_pedidos

5

vaciar y avisar

y la pantalla entera se
acomoda sola

```
// lo que devuelve leer el formulario:
{ [k: string]: FormDataEntryValue }
// TypeScript NO sabe que hay nombre, email,
pago
```

```
// la ADUANA 3: en UN solo lugar, y se ve
const leerCliente = (f: HTMLFormElement):
Cliente =>
  Object.fromEntries(new FormData(f)) as unknown
as Cliente
```

⚠ La validación nativa es **comodidad, no seguridad**: cualquiera borra el required en DevTools. · Y el orden importa: primero se guarda el pedido, después se vacía el carrito.

LA TIENDA COMPLETA · Y AHORA TIPADA

Siete archivos, **un contrato**

tipos.tsqué ES
cada cosa**datos.ts**

el plan B

api.ts

habla y traduce

carrito.ts

piensa

formato.ts

un precio

ui.ts

un producto

main.ts

orquesta

`tipos.ts` **no genera una sola línea** de JavaScript... y sostiene a los otros seis.

0 KB en el navegador

la única mejora de calidad de código que no cuesta rendimiento

`npx tsc --noEmit` = revisar todo el proyecto sin generar nada. Dos segundos. **Eso corre antes de cada entrega.**

SEIS IDEAS, EN EL ORDEN EN QUE LAS VIMOS

Resumen

los tipos desaparecen

es un compilador: el navegador ve JavaScript

interface = la forma

estructural, no nominal — la gran diferencia con Java

| ? & =>

uniones, opcionales, intersecciones y tipos de función

narrowing

el compilador lee tus `if` y estrecha el tipo solo

any vs unknown

apagar TypeScript, o que te obligue a mirar

las tres aduanas

API · localStorage · formulario

Anotar un tipo no valida nada.

TypeScript te cuida de vos mismo, **no del mundo exterior.**

TODAS SOBRE SU PROYECTO · SI COMPILA SIN ERRORES, VAN BIEN

5 tareas

FÁCIL

Tipar lo que quedó suelto: corran `npx tsc --noEmit` y déjenlo **en cero**. Quedaron `masCaroDe`, `fichaProducto` y el evento `.target` de los listeners.

INTERMEDIO

El estado de carga, como tipo: `let estado: EstadoCarga + pintarEstado()`. Si agregan un estado y se olvidan de dibujarlo, que **el compilador lo diga** (pista: `never`).

INTERMEDIO

contarPorCategoria, tipada: "Laptops (5)" en la lateral. El retorno se escribe `Record<string, number>`.

DIFÍCIL

El detalle del producto: `fetch a /products/{id}`, ampliar `ProductoAPI`, y usar `descripcion?` — que no los va a dejar sin preguntar.

DIFÍCIL

"Mis pedidos", con su aduana: leer `techcart_pedidos` con `unknown` + comprobar, escapar el nombre del cliente, y `pintarPedidos()`.

Y ahora sí: React

```
interface PropsTarjeta {  
  producto: Producto  
  onAgregar: (id: number) => void  
}  
  
export default function Tarjeta({ producto, onAgregar }: PropsTarjeta) {  
  const [abierta, setAbierta] = useState<boolean>(false)  
  return <article>{producto.nombre}</article>  
}
```

Lo **celeste** ya lo saben leer entero: la interfaz de props, el tipo de función, el genérico. Lo único genuinamente nuevo es lo gris: que una función **devuelva HTML**. Eso se llama **JSX**.

Nosotros escribíamos `pintarCatalogo`, `pintarCarrito`, `pintarTitulo...` y había que acordarse de llamarlas. React hace eso y nada más: cambiás el dato, él repinta.